



Using DNS Patterns for Automated Cyber Threat Attribution

Cristoffer Leite
c.leite.da.silva@tue.nl
Eindhoven University of Technology
Eindhoven, Netherlands

Jerry den Hartog
j.d.hartog@tue.nl
Eindhoven University of Technology
Eindhoven, Netherlands

Daniel R. dos Santos
Fore Scout Technologies
Eindhoven, Netherlands

ABSTRACT

Linking attacks to the actors responsible is a critical part of threat analysis. Threat attribution, however, is challenging. Attackers try to avoid detection and avert attention to mislead investigations. The trend of attackers using malicious services provided by third parties also makes it difficult to discern between attackers and providers. Besides that, having a security team doing manual-only analysis might overwhelm analysts. As a result, the effective use of any trustworthy information for attribution is paramount, and automating this process is valuable. For this purpose, we propose an approach to perform automated attribution with a source of reliable information currently underutilised, the DNS patterns used by attackers. Our method creates recommendations based on similar patterns observed between a new incident and already attributed attacks and then generates a list of the most similar attacks. We show that our approach can, at ten recommendations, achieve 0.8438 precision and 0.7378 accuracy. We also show that DNS patterns have a short lifespan, allowing their utility even in more recent knowledge bases.

ACM Reference Format:

Cristoffer Leite, Jerry den Hartog, and Daniel R. dos Santos. 2024. Using DNS Patterns for Automated Cyber Threat Attribution. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024)*, July 30–August 02, 2024, Vienna, Austria. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3664476.3670870>

1 INTRODUCTION

Modern networked systems are constantly under the threat of cyber attacks. In this scenario, Cyber Security Incident Response Teams (CSIRTs) monitor their networks for possible threats. They usually employ a Network Intrusion Detection System (NIDS), which observes the network and applies counter-measures for malicious activity, with the CSIRT analysing its output for more precise incident response. Part of the analysis of malicious activity detected by the NIDS consists in attributing these actions to a responsible threat actor. This process is known as cyber threat attribution.

When attacked, a victim would like to know the perpetrator. Performing such threat attribution can have several benefits. In the first place, it enables an organisation to understand its adversaries, their modus operandi and their objectives, which helps to prepare its

defence against future cyber attacks [52]. More generally, attackers leverage Internet anonymity to conduct surveillance, probes, and attacks without attracting attention or being exposed [15].

Removing or reducing this anonymity might thus help discourage attacks. This situation also applies to nation-state actors. When held accountable for their actions, states make a different assessment than an attacker who can operate in complete anonymity [13], and legal accountability against them only comes after cyber attacks are attributed [14]. Thus, as a country, attributing an attack to another country makes you less attractive as a target.

Performing threat attribution, however, is challenging. Analysts need to compare observed information to attacks reported by other entities in Cyber Threat Intelligence (CTI) reports. The management of several incidents and alerts gets worse when mixed with the plethora of Open-Source CTI (OSCTI) that analysts need to procure to investigate an attack. Although there is a growth in applying Machine Learning techniques to automate and reduce this often manual effort [4, 20], the lack of context is still considered one of the biggest challenges for using these capabilities [9].

Attackers may also employ misguiding approaches to avoid identification or even induce analysts to wrong conclusions regarding the attack's responsibility [46]. Moreover, the trend of malicious services provided by third parties, such as an Initial Access Brokers (IABs) [34] or with a Malware as a Service (MaaS) [10], also challenges attribution. For instance, when different actors may use the same service as part of their attack. In these cases, part of the evidence might point to the provider rather than to the attacker.

As such, threat attribution must be the result of a combined and incremental analysis [39] around multiple factors [44], and this analysis needs to use trustworthy information, i.e. types of evidence that are hard for an attacker to hide or manipulate. Suggested candidates include the Tactics, Techniques and Procedures (TTPs) used by the attackers [45], which Cloud Services hosted it [1], information regarding any Command and Control (C2) Infrastructure used [18] and the Domain Name System (DNS) patterns present in the domains used by the attackers [46]. As the attacker tries to hide as much information as possible, any available trustworthy information should be used effectively.

In this paper, we focus on attribution based on DNS patterns from different phases of an attack. These patterns include DNS records, information about host providers and passive DNS services [25]. As a potential source of trustworthy information that has thus far received limited attention, it could offer information complementary to existing attribution approaches. We aim at an automated approach for extracting as much useful and actionable information from the DNS patterns as possible in a way that allows it to be integrated into other approaches. By “actionable”, we mean that an analyst can understand why a certain attribution is recommended.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ARES 2024, July 30–August 02, 2024, Vienna, Austria

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1718-5/24/07

<https://doi.org/10.1145/3664476.3670870>

To this end, we create a recommendation system that, for a new incident, suggests the most similar incidents from a collection of already attributed attacks based on their DNS patterns. Each step of an attack receives a cluster tag. Then, it creates an incident similarity score based on the similar patterns shown between domain-related alerts involved. Considering patterns in different steps, rather than just individual domains, helps attribution in cases where phases are shared amongst attackers, such as with IABs and MaaS mentioned above.

Our main contribution is thus an automated technique utilising DNS patterns of incidents to perform attribution with high accuracy. In doing so, we also provide a method to extract actionable information from domain-related alerts and find similar incidents. Additionally, we analyse the lifespan of DNS patterns. Besides their usefulness for deploying the approach, these results may also offer some insight into the temporal behaviour of domain usage by attackers.

The remainder of this paper is organised as follows. The next section describes the terminology, setting, scenario and motivation in more detail. Section 3 provides a conceptual overview of our approach. Section 4 details an implementation of our approach, while Section 5 provides related experiments and discussion of the results. Section 6 provides conclusions.

2 BACKGROUND

In this section, we introduce the current scenario where attack attribution happens with some motivations for our work, then we present the current state of the art for threat attribution approaches and those that are more similar to ours.

2.1 Scenario and Motivation

A NIDS monitors network traffic and identifies events that exhibit suspicious or known bad behaviour. Observed network events classified as possible malicious activities are flagged as alerts by the NIDS. A collection of related alerts is an incident, and an incident with known malicious behaviour is an attack. Consequently, alerts encapsulate a wealth of information about observed network attacks, i.e., the evidence concerning an attack.

The malicious entity responsible for an attack is called an actor or an attacker. The term actor can refer to an individual or a group. Advanced Persistent Threats (APTs) are actors with a highly sophisticated level of expertise and resources, sometimes managed (or supported) by a government [22]. Multiple attacks performed by actors in close periods against a specific target profile and displaying similar adversarial behaviour are called campaigns [5]. A campaign can also denote an infrastructure reused by attackers in different attacks [11].

Threat attribution aims to identify the actor responsible for an attack or to place an attack within a campaign. A CSIRT uses multiple types of evidence from an incident to analyse and possibly attribute the attack to an actor. But as mentioned before, attackers can employ misguiding approaches to avoid identification or even induce analysts to wrong conclusions regarding the attack's responsibility. Using the most trustworthy information is thus paramount to correctly attribute an attack [46].

Different pieces of reliable information can help map an attack and its attacker. For example, the TTPs show the approaches used by the attackers in different phases, delineating their behaviour [45]. Information about the C2 Infrastructure can give a good indication on the central point orchestrating an attack [18]. We focus on another reliable type of trustworthy information, the DNS patterns displayed by the attackers [6, 25].

DNS patterns contain the information related to domain registration and attack hosting. For attribution using DNS patterns, it is imperative to account for the different considerations attackers make for domain creation in each attack phase and the existence of exploitation services provided by other parties.

Each phase of an incident can have unique characteristics in its domains. The initial attack phases often require more convincing domain names to lure victims into it as they need to appear as legitimate or trustworthy websites. Intermediate stages apply domain names produced by a Domain Generation Algorithm (DGA) more frequently, as they are helpful contact points spawnable on the fly [36]. The final stages of an attack vary depending on its goals. For instance, some data ransoms use dark web domains and similar ones, such as *.onion* domains from the Tor Network, to host the information presented to the victim [50], while other attacks use legitimately looking domains over the DNS protocol itself as a covert channel for data exfiltration [30].

Malware as a Service and other third-party services for exploitation also impact the domains used in different phases of an attack. Even a few years ago, attackers would already employ Exploit Kit (EK)s supplied by a separate entity to execute their attacks [3]. Now, Ransomware as a Service (RaaS) [23, 29] and similar types of MaaS [10] are common and profitable products provided by malicious actors in black markets. Providers of such malicious software give their clients pre-configured servers running their applications, and these services very often include pre-registered domains [16]. As a consequence, domains hosting these services follow other patterns.

These cases reinforce the idea that DNS pattern analysis needs to consider domain patterns changing across different phases of an attack. Using DNS patterns for attribution should account for these phases by looking at an attack as a composition with varying DNS patterns.

Before looking at how we consider compositions (Section 3.2) of DNS patterns (Section 3.1) for threat attribution, we first discuss related work in the next subsection.

2.2 Related work

In this section, we first present an overview of the most recent works regarding the automation of threat attribution. Then, we focus on the recent works that apply some form of DNS patterns for detection or attribution.

2.2.1 Threat Attribution. Using existing knowledge provided by CTI to do Threat Attribution is common. Roughly speaking, proposed techniques use artefacts from host- or network-based data for attribution. Some methods use only host-based data, such as OpCode and ByteCode sequences and Header information observed in executables [17, 40], specifically for malware attribution.

Multiple approaches incorporate both host- and network-based data for attribution [33, 38] by characterising the overall behaviour of each threat actor and then leveraging this knowledge to safeguard a system against them. Some works do that in specific scenarios or sectors, such as Cyber Physical Systems (CPS) [21] and Financial Technology [32], to reduce the effort and make the solution more tractable. Some others concentrate their efforts on the more reliable types of data like TTPs [24, 32].

2.2.2 DNS Patterns. For network-based attribution, we focus on recent works that use some form of DNS Pattern in that direction. This information includes data collected from domain WHOIS, passive DNS and, quite often, C2 Infrastructure. We also investigated the leading works that use this information for the detection of attacks.

Most existing works focusing on automating the use of DNS patterns only consider some limited scenarios. For instance, many proposals use DNS information to identify domains as malicious [2, 41, 55]. Notably, Bilge et al. [6] analyse features extracted from passive DNS information to characterise and identify standalone domains involved in malicious activities. Their classifier can label domains as either benign or malicious. One advantage of their method is being broad enough for multiple cases. Sebastián et al. [43] expand on that concept by using additional types of features, including some extracted from TLS certificates and web content. Then, they create a content attribution which can determine the specific ownership of some domains. Thomas and Mohaisen [49] cluster domains based on their traffic volume and lookup patterns to attribute single domains to a malware family.

These approaches assess attackers' conventions while registering domains throughout the entire attack. Comparing the domain patterns as a monolithic block has its merit in identifying broad patterns, but it can fail as the sophistication of attacks increases and the patterns change per phase. It is necessary to be able to identify different patterns in multiple phases of the attacks.

The approach that seems closest to our proposal is the work by Li [25]. The author of that work collects sets of malicious DNS-IP pairs and additional WHOIS information. Then, they use a link analysis tool (Maltego) to see matching ones based on their patterns.

Although supported by a visualisation tool, the matching of patterns is still mostly manual. That limits the work as a standalone approach and makes integration with other analyses difficult. Besides that, the work also used only a couple of samples.

To the best of our knowledge, no work presents a method of using DNS Patterns for threat attribution in such a way that (1) identifies different patterns in multiple attack phases, (2) in an automated manner, and (3) can directly integrate with other techniques. Using this information in a broader view to attribute an attack has not been addressed so far, so we borrow ideas from these different approaches to deliver a methodology that addresses this issue.

3 METHODOLOGY AND DEFINITIONS

Our objective is to use DNS patterns to assist cyber security experts in attributing attacks during their analysis. We focus on investigation support by providing information on known attacks, their respective actors and their patterns overlapping with new incidents.

Our method outputs the known attacks most similar to a new incident based on their DNS patterns. Each known attack presented includes at least its given attribution (e.g. to an actor) and a list of domains used by it. For transparency and actionability, the method's output also details the DNS pattern similarities between the new incident and the known attacks, their score, and the features extracted from the observed domains.

Figure 1 shows the main phases of our approach. Most of the first phase runs before the analysis of a to-be-attributed attack happens. This phase prepares the knowledge dataset used to attribute new incidents. The second phase takes place during the analysis of an attack for attribution.

The first phase is the Domain Pattern Analysis. It clusters domain-related alerts based on their DNS similarity, resulting in cluster tags for domain-related alerts (those without a cluster we call *untagged*). In the second phase, called Incident Similarity Scoring, each incident receives a *Cluster Tag Set* consisting of the tags on the domain-related alerts within the incident. Then, we use these tag sets to compute a pair-wise *Incident Similarity Score* as the intersection of cluster tags between known attacks and the new incident. Finally, by ranking known incidents based on their similarity to a new incident, we present the most similar ones to the analyst.

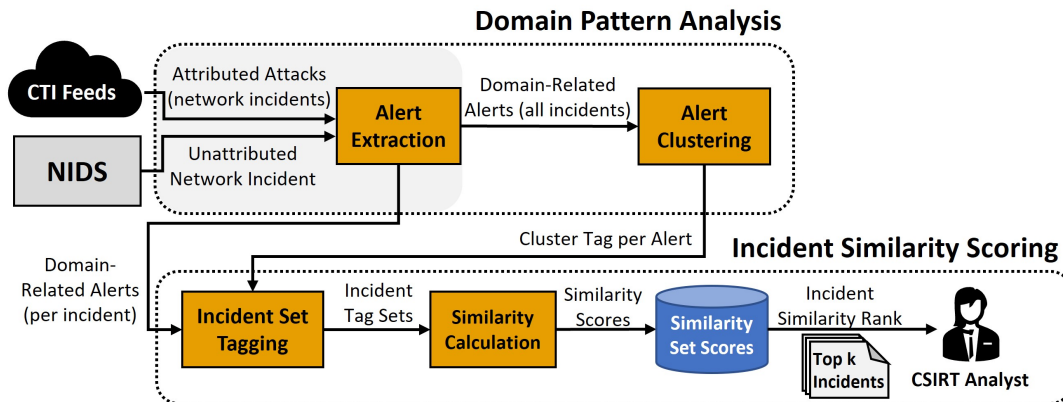


Figure 1: The Architecture of our Solution

The following sections will present the definitions of our approach and explain each phase in more detail. They also contain motivations behind some of the choices made.

3.1 Domain Pattern Analysis

Networked events observed by a NIDS may contain web domains in their event information. These domain-related alerts are the foundation of our analysis. As we evaluate domains regarding their features, in our formalization, we define domain-related alerts based on their DNS features. We will assume that an alert contains exactly one web domain, discarding alerts that are not domain-related and splitting any that mention multiple domains.

Definition 3.1 (Domain-Related Alert). A domain-related alert a is a tuple $a = (r_1, \dots, r_m)$ of its domain-related features, with each feature r_i , ranging over a metric space V_i , capturing some meaningful aspect of a . $\mathcal{A}$ denotes the domain of all alerts.

As mentioned, we only consider alerts that are domain-related. Alerts with high similarity in their DNS features indicate a likelihood of them having a similar origin [6] alerts include the domain name (e.g. *example.com*), the registrar and the host service provider. Section 4.2.1 shows more concrete examples.

Note that each feature domain V_i is defined as a metric space. Thus, for each V_i , a distance function (or dissimilarity) $d_i : V_i \times V_i \rightarrow [0, 1]$ captures how dissimilar two alerts are concerning their feature values in r_i . Sometimes we give the similarity function $1 - d_i$ instead of defining d_i directly. With distances per feature given, we define the distance $D : \mathcal{A} \times \mathcal{A} \rightarrow [0, 1]$ between domain-related alerts as their weighted average:

$$D((r_1, \dots, r_m), (r'_1, \dots, r'_m)) = \sum_{i=1}^m w_i * d(r_i, r'_i) / m \quad (1)$$

Alerts are typically not analysed individually. Instead several alert-correlation techniques exist that can group them together. We assume such a technique is in place and refer to a group of correlated alerts as an *incident*. If evident malicious activity is present in an incident, it is classified as an attack. In the context of this methodology, the only incidents considered are already known to be attacks and the terms are used interchangeably.

Definition 3.2 (Incident). An incident I is a set $I = \{a_1, \dots, a_n\}$ of instances of domain-related alerts.

An attribution indicates the adversary activity group (actor or group) responsible for executing an attack or its related campaign. We call an incident *attributed* if some attribution for it exists and *unattributed* otherwise.

Attackers are known to be increasingly less dependent on static domains. Techniques like DGAs [47] make it easier for them to register and apply an arsenal of different sources to avoid signature-based blocking and prevent other forms of static investigation.

Analysing domains in alerts must take these factors into account during attribution. Thus, instead of looking at equality between domains, we use a concept of similarity between alerts [49]. We already defined a notion of distance between alerts with Equation 1, so we use this notion to perform clustering to find similar alerts from multiple incidents.

Given a set $T = \{a_1, \dots, a_m\}$ of alerts instances from multiple incidents, a clustering function U aggregates similar alerts while separating dissimilar ones in different clusters based on the distance $D(a, a')$ of each pair $(a, a') : T \times T$.

Definition 3.3 (Alert Clustering). A clustering $U : T \rightarrow \mathbb{N} \cup \{-1\}$ assigns a cluster tag to any alert in T , with $U(a) = -1$ indicating that an alert a is unclustered.

An alert tag represents the cluster an alert belongs to. Those with the same tag (except -1) belong to the same cluster, indicating that they have a high similarity in their domain-related features. We explore different clustering techniques as implementation options in Section 4.1.1, with a comparison between their effectiveness in Section 5.4.

Figure 2 shows an example with five separate attacks related to the Cerber malware family (v4) within one month [35]. These samples relate to the Pseudo-Darkleech Campaign alternating between using RIG EK and RIG-V. A colour distinguishes each attack. The arrows represent the progression of the attacks over time, and the symbols represent the domains in the alert instances and what they host. There, alerts from three different attacks (03-10, 04-10 and 05-10) have a matching cluster tag α in their attack execution phase, while a similar thing also happens for the last phase of all attacks with a cluster tag β .

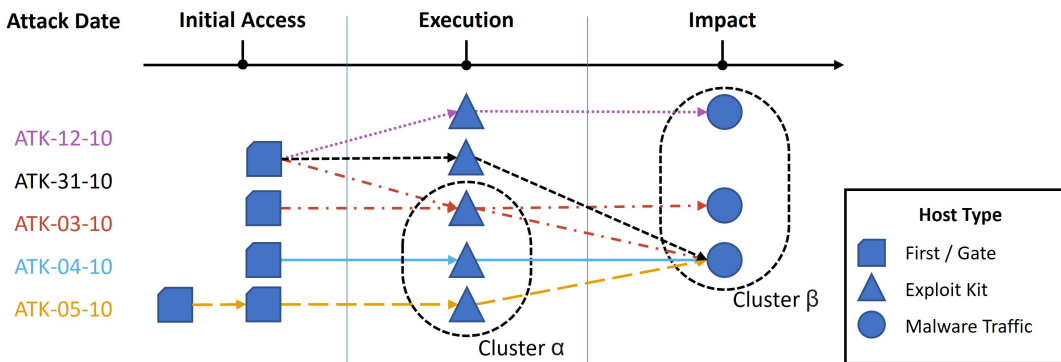


Figure 2: Different Attack Progressions with the Cluster Tags from their Alert Instances

	Indicator	Description	Resolved IP	Host	Registrant
04-10	[hidden website]	Compromised website			
	sd.contentwize.com	Rig EK	194.87.239.147	Startup Maint.	suparno2k@yahoo.com
	ffoqr3ug7m726zou.zxtezv.bid	HTTP Traffic - Cerber	173.254.231.111	Quadranet	
	Indicator	Description	Resolved IP	Host	Registrant
05-10	[hidden website]	Compromised website			
	zpcjrwyf.ddnsking.com	Redirect/Gate	83.217.27.178	DATAPRO, LLC	Privacy by No-IP.com
	vd.dollarfordiamond.com	Rig EK	195.133.201.61	Startup Maint.	suparno2k@yahoo.com
	ffoqr3ug7m726zou.zxtezv.bid	HTTP Traffic - Cerber	173.254.231.111	Quadranet	

Figure 3: Overlaps Between Two Attacks with the Cerber Ransomware

The example in Figure 2 shows clustering effectiveness in matching different domain patterns from related attacks. Figure 3 depicts, in more detail, two of these attacks (04-10 and 05-10) overlapping in different phases. Different attacks can use the same hosts and domains in similar phases, as in the last phase in the example. However, these attacks can still differ on other hosts and the domains involved, as in the second phase of the attacks. As a side note, the domains involved in the initial phase should not form a cluster because they are just compromised websites, i.e. they were legitimate websites that attackers stole for their attack.

To allow matching related attacks while still distinguishing different types of attacks that share domains, we will look at classifying the sets of tags involved in incidents. The next section explains how we combine the cluster tags, representing the DNS patterns, from the entire incident to help in the attribution.

3.2 Incident Similarity Scoring

It is crucial to consider that analysing domain patterns in single alerts alone can be misleading. As referenced before, using third-party services to execute attacks is common. When these services are used in an attack, analysing a single alert in isolation could point at the malicious service providers rather than the attackers responsible for the whole attack. While identifying the source and use of these services is valuable, it is not sufficient for attributing the entire attack.

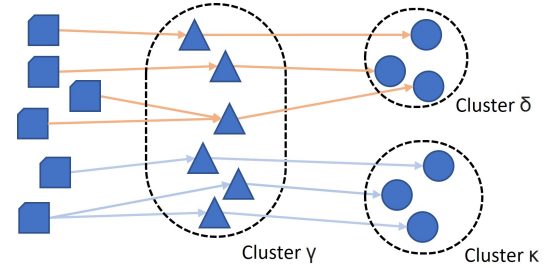
As mentioned in Section 2.1, an attack is a combination of different phases with possibly different requirements for domain creation. Analysing groups of domains involved as a composition of different steps helps in the attribution even when third-party services are used [25]. In that direction, the tags provided by the Domain Pattern Analysis serve as the starting point for the incident set tagging. This phase attributes to the entire incident a set of cluster tags received by its domains.

Definition 3.4 (Cluster Tag Set). Given a clustering U , the Cluster Tag Set $I.t$ for an incident I is the group of cluster tags received by its clustered alerts in the form $I.t = \{U(a) \mid a \in I \wedge U(a) \neq -1\}$.

A *Cluster Tag Set* summarises how similar the alerts in a new incident are to the ones within known attacks. Because each tag represents a cluster of similar alerts in different phases, the Cluster Tag Set shows the intersections of clusters between attacks. Consequently, unclustered alerts do not contribute to this set, as they are not in a group of similar alerts. Note that, because it is a set, repeating tags do not need to be counted.

The example presented in Figure 2 shows intersecting cluster sets between domains of incidents, indicating matching patterns. In that case, the Cluster Tag Sets are $\{\beta\}$ for the two first incidents and $\{\alpha, \beta\}$ for the three remaining ones. Note that in this example, the phases above it (Initial Access, Execution and Impact) are just interpretations of the clusters, as they are not inherently part of the cluster sets.

It is valuable to know when different actors use a service from the same malicious provider, but differentiating between these actors is also helpful. Figure 4 shows an example where the second phase of attacks from different groups matches and thus creates a cluster of similar alerts. In that case, the methodology already gives information about a malicious service provider. But the DNS patterns diverge again in the last phase when actors deploy their own infrastructure for the post-infection phase. In this specific case, the pattern intersection happens in the part where they deploy the Nuclear EK, which was known for its MaaS infrastructure used by numerous actors [7] as in the example.

Figure 4: Visualisation of an Infrastructure (γ) with Nuclear EK Shared by Different Actors in a Malware-as-a-Service

Cluster sets have the potential to both link related attacks and distinguish between different ones, even if they share a domain, for instance, due to the use of services from a malicious third party. Because of that, we use the similarity between cluster sets to rank attacks similar to a new incident being analysed.

Given the Cluster Tag Set from a new incident, it is now necessary to identify the most similar known ones. Considering a set Y including all known incidents and the new one, we look at the overlapping cluster tags from any two incidents to find similar attacks. The set of Y of all incidents is thus a metric space, and the Incident Similarity Score given by a function S is based on the intersection of cluster tags in two incidents.

Definition 3.5 (Incident Similarity Score). Given a set of incidents Y with a clustering, for any incidents $I, I' \in Y$, we define the Incident Similarity Score $S : Y \times Y \rightarrow [0, 1]$ as the Jaccard Index similarity between their cluster tag sets:

$$S(I, I') = |I.t \cap I'.t| / |I.t \cup I'.t|$$

If both $I.t$ and $I'.t$ are empty, $S(I, I') = 1$ if $I = I'$ and 0 otherwise.

The Jaccard Index is a widespread metric used to calculate set similarity [27]. It computes the similarity as the ratio between the cardinality of the overlapping membership and the total cardinality of the union of both sets. This score gives adequate information about the DNS patterns in two incidents to make them comparable by reflecting the percentage of cluster intersections between incidents.

With the similarity scores computed between a new incident and all known attributed attacks, it is now possible to create a list of suggested attributions for the new one. We only consider two incidents I and I' similar if they have at least some overlap in their Cluster Tag Set, i.e. $S(I, I') > 0$. Finally, the list only includes the k most similar incidents ordered by their score. The final result is a recommendation list of incidents with similar DNS patterns as the new one.

Definition 3.6. For a set Y with a clustering, an $I \in Y$, and a integer k , a recommendation system $\Omega(I, Y, k)$ is a sorted list of the k most similar incidents to I in $Y \setminus \{I\}$ (or all similar ones if there are less than k), i.e. $\Omega(I, Y, k) = (I_1, \dots, I_m)$ with:

- $S(I, I_1) \geq S(I, I_2) \geq \dots \geq S(I, I_m) > 0$
- $S(I, I_m) \geq S(I, I')$ for all $I' \in Y \setminus \{I, I_1, \dots, I_m\}$.
- $m \leq k$ and $m < k \Rightarrow (S(I, I') = 0 \text{ for all } I' \in Y \setminus \{I, I_1, \dots, I_m\})$.

By taking a new incident I and a set of attributed incidents $Y \setminus \{I\}$, a recommendation system $\Omega(Y, I, k)$ gives the analyst a context in which to examine a new attack and evaluate possible parallels with known ones. The threshold k limits the amount of information for the analyst to process.

Given the definitions for our methodology, the following section presents an implementation of this approach.

4 IMPLEMENTATION

This section shows our implementation of the methodology proposed in the previous section and discusses the reasoning behind some decisions made during the implementation.

4.1 Clustering Choices

Our approach requires a clustering technique (Definition 3.3) to find similar domains. In implementing this clustering, we face several choices: which features to capture in the data to cluster, what clustering function to use in finding candidate clusters and how to select clusters of sufficient quality from the candidates.

4.1.1 Clustering Functions: For some attack phases, attackers deploy tailored domains to make it harder for victims to identify them as malicious. These domains should not be in a cluster, so a clustering technique capable of handling noise is preferable. We select three widely used methods that can support this. They are variations of Density- and Hierarchical-based clustering.

The first technique is the Density-Based Spatial Clustering of Applications with Noise (DBSCAN), which searches the data for high-density areas, define their cores as cluster centres and, based on a distance ϵ between two entities, expands their radius from that point [42]. The second technique is an extension of DBSCAN called Hierarchical DBSCAN (HDBSCAN). This approach creates a cluster hierarchy, identifies which clusters are stable when the density changes, and then prunes the hierarchy to keep only the stable clusters [28].

The last technique is the Balanced Iterative Reducing and Clustering using Hierarchies (BIRCH), which first creates a summary with each dense region, referred to as a Clustering Feature (CF). Then, it organises these regions into a CF Tree, where each sub-cluster is a leaf node. If the number of clusters in a leaf surpasses a threshold (called the branching factor), it splits it into new leaves [54]. We test which of these techniques performs better under different circumstances in Section 5.

Each clustering technique requires tuning of one or two parameters. We performed this tuning using the metrics discussed in the next section. For DBSCAN, we set the ϵ parameter to 0.5. For both HDBSCAN and DBSCAN, we use a minimum neighbourhood of 2. BIRCH has a cluster distance threshold which we keep at the default of 0.5, and a ‘number of clusters’ parameter, which we set to 700 clusters. The number of clusters was decided based on our observations of the best results achievable. But interestingly, this is close to the average number produced by DBSCAN and HDBSCAN in our tests.

4.1.2 Cluster Quality: The clusters of domains are the foundation of our method. They aid the attribution by helping analysts discern between different patterns. It is valuable to validate how much this quality impacts the attribution recommendation system [19]. As such, we consider how the inclusion of a cluster quality filter affects the results of our experiments.

We use the Structural Quality of a Cluster [12] because it gives the utility of a clustering technique. With it, a cluster has low structural quality if too many features have a low average similarity between entities. In our case, that means multiple features with little chance of having similar DNS patterns in a cluster. We limit the clusters considered for the Tag Sets to only those with higher quality when this improves the results of the techniques used in our implementation.

4.2 Features and Metrics

4.2.1 Features: We selected features that can provide as much complementary information about DNS patterns as possible. In addition, we avoided using information with insufficient availability on multiple services, such as the ones hidden by attackers abusing WHOIS privacy [8, 26].

Table 1 shows the features used in our implementation with an example for each, their source, and the distance function for their metric space. Some of these features were directly obtainable from the alerts, but we also used domain-related information that could be extracted from other sources, creating additional features. In all cases, we selected features representing a meaningful aspect of an alert as per Definition 3.1.

Table 1: Features Selected, Including an Example, their Source and the Distance Metric

Feature	Example	Source	Metric
Domain	'google.com'	Alert	Levenshtein
Subdomains	[mail, docs]	Alert & Passive DNS	Jaccard
Email User	'user'@	WHOIS History	Levenshtein
Registrar (ID)	'MarkMonitor' (ID292)	WHOIS or Passive DNS	Levenshtein*
Host Provider (ASN)	'Worldstream' (AS49981)	Passive DNS	Levenshtein*

We leverage data from Passive DNS and historical WHOIS services to enrich our dataset. Passive DNS has proven usefulness in both identifying [6] and attributing [43] malicious domains. WHOIS data is another valuable and often used source that has good results in previous works [48]. But, as some attacks are a few years old, these tend to be unavailable on most services. We resorted to services providing historical WHOIS data, as they store information from when these attacks happened.

4.2.2 Feature Metrics: For string values, which is the case for most features, we use the Levenshtein Index [31] as the distance for their metric space. The Levenshtein Index, also known as edit distance, calculates how many insertions, deletions, and substitutions are necessary to transform one string to another, scaled from 0 to 1. This index is applicable even when the strings have different lengths. For both *Host Provider* and *Registrar*, they get maximum similarity when they have, respectively, the same Autonomous System Number (ASN) or Registrar ID from the Internet Assigned Numbers Authority (IANA). Otherwise, they receive the Levenshtein Index from their strings. For the feature *Email User*, it means it only calculates the Levenshtein distance using the username part of an email.

The *Subdomains* feature is a set of subdomains observed by passive DNS for a domain. For example, observing mail.google.com and docs.google.com through passive DNS creates the associated subdomains set as [mail, docs] for google.com. We use the Jaccard Index to calculate the distance for the Subdomain metric space. We use subdomain sets because, together with the additional WHOIS information, they can indicate when there is DGA similarity [48].

4.2.3 Performance and Timing. The most time-consuming part of our methodology is computing the distances used in the clustering. This step's complexity is quadratic in the number of alerts and takes a few hours in our test machine (8th generation Intel Core i7, 32GB of RAM) for a dataset containing 12,000 alerts and using one thread for each feature. Note, however, that most of these can be pre-computed. When a new incident arrives, the approach only needs to compute the distance of a new incident to each existing one. This live part of the computation is a linear number.

With the precomputation done, the rest of the attribution (clustering, filtering, pattern computation and ranking) takes only a few minutes on our test machine. As this is done only on incidents confirmed as malicious rather than on all new incidents, we do not consider this a prohibitive amount of computation time. Also, the pattern lifetime results in Section 5.4.3 show how possible it is to reduce the dataset and, consequently, the computation times. We, therefore, did not do optimizations nor measure performance.

5 EVALUATION

5.1 Dataset

We use the datasets from open threat reports provided by Malware-Traffic-Analysis¹ and Palo Alto Unit42², with enriched data from AlienVault Open Threat Exchange (OTX)³.

We use a dataset with more than 2,000 previously reported attacks for our tests, with more than 12,000 malicious domains included in the traffic generated across the attacks. It includes attacks happening between 2014 and 2024 with information regarding actors, campaigns and the software involved.

We give each actor a unique identifier. Cybersecurity institutes use different naming schemes for actors. We apply a standardisation to the actor information extracted when necessary based on the open dataset of *APT Groups and Operations*⁴ to avoid mismatches.

The dataset contains attacks combining mostly Ransomware, Botnets, Phishing Campaigns, Malspams and multiple Exploit Kits. Although the name of the software involved is not mandatory for our methodology, it can provide more evidence later during the attack analysis, increasing its actionability. This information was available on most attacks, so we used Alienvault's naming map to standardise the names of malicious software.

On each attack extracted from a report, a request to a malicious domain is equivalent to a domain-related alert in the NIDS. These are the basic features already available: the domain and occasionally one or more of the subdomains. We enrich our dataset with the additional features mentioned in Table 1. AlienVault OTX is also our source for Passive DNS information. Whoxy provides the WHOIS history from when the attack happened. Both cases already provided structured data, so for both the Registrar ID and the AS Number, there was only the need to map these identifiers to the name registered with IANA. For the emails, we filtered those masked by a privacy service. We did the same in cases with an abuse notification in the place of the email, but using WHOIS history already reduced these cases to less than 3% of the dataset.

Almost all domains extracted from the Threat Intelligence Platform (TIP)s had a description related to their use given by the threat report. For example, in the sample shown in Figure 3, two domains hosted RIG EK and two hosted Cerber ransomware payment pages. This information is useful later when we show the related incidents to an analyst, so we add them as labels to each alert by parsing their content.

¹<https://malware-traffic-analysis.net/>

²<https://unit42.paloaltonetworks.com/>

³<https://otx.alienvault.com>

⁴<https://apt.threattracking.com>

5.2 Evaluation Metrics

Our recommendation system results in a sorted list of k similar incidents. An incident is correctly classified if its actor is within the top- k similar incidents recommended. Thus, for a given k there are three possible outcomes; *No recommendation* (NR) happens when it finds no sufficiently similar incidents. A *true recommendation* (TR) is when the actual actor is the perpetrator of one of the top- k similar incidents. And a *false recommendation* (FR) is when none of the top- k similar incidents has the actual actor.

We have a ‘multi-class single-label ranking with rejection’ problem. As the classes and, likely also, the cost for mispredictions are relatively balanced, accuracy should be a suitable quality indicator [51]. Top- k accuracy is a metric commonly used to score ranking algorithms, as it evaluates how often a method makes the correct prediction in the entire sample within the top- k predictions [53]. For our case, it shows the percentage of incidents that get a true recommendation ($\#TR / (\#TR + \#FR + \#NR)$).

On the ones without a true recommendation, it is relevant to know whether they have any recommendations at all. Thus, we also dissect top- k accuracy into completeness, i.e. the percentage of incidents that get any recommendation ($\#TR + \#FR / (\#TR + \#FR + \#NR)$) and the percentage of recommendations that are true ($\#TR / (\#TR + \#FR)$), which we’ll call top k -precision. Any non-empty list is a recommendation, even if it contains less than k entries. Thus, completeness is constant across different k . Note that top- k accuracy = top- k precision * completeness. We use the terms precision and accuracy from binary classification, as they yield the same formulas. Precision is also known as non-rejected accuracy in some settings.

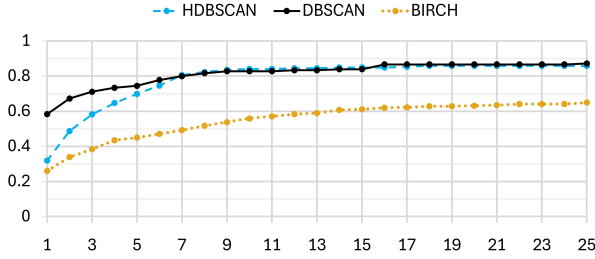
As attackers evolve and adapt their infrastructure for new attempts, the DNS patterns they use between attacks will eventually change. Thus, we also evaluate how long it takes for them to change by measuring the distance in days between a new incident and its furthest match on the recommendation list. Here, we say two incidents match if they are similar (see Definition 3.6) and have the same actor.

5.3 Tests

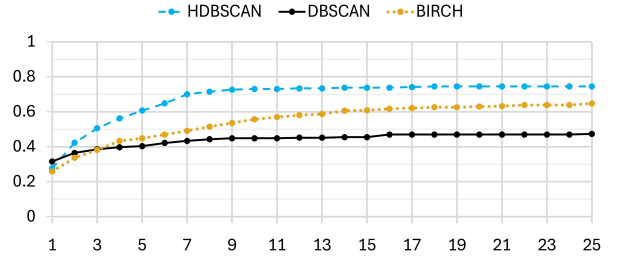
We first evaluate the performance of the selected clustering algorithms. This test confirms the approach’s feasibility and provides insights into suitable clustering techniques. Next, we measure the impact of cluster quality filtering and the combination of different techniques. The results help optimize the configuration. Finally, we investigate the usual lifespan of a DNS pattern. This information supports analysts in configuring their setup, especially concerning the dataset of known attacks. It also provides insights into the use of an infrastructure by attackers.

We use a tenfold cross-validation in all tests related to the techniques with a 75%/25% split into two groups. The first group, containing 75%, is treated as the group of known attacks. The second group, with the remaining 25%, is treated as the group of new incidents. For each incident in the second group, we execute the attribution recommendation using the group of known attacks as our knowledge base. We take the average result for each metric described above from the ten repetitions with different splits.

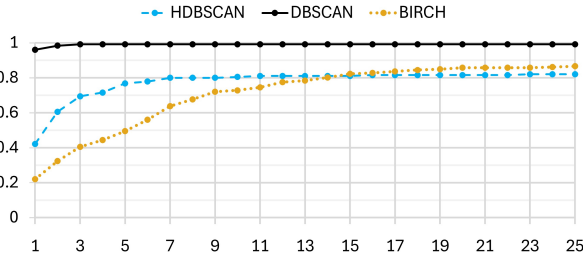
Based on the performance of the techniques, we suggest a combination of the best ones for a new recommendation system. Finally, to help other analysts create their implementations, we investigate the usual lifespan of a DNS pattern.



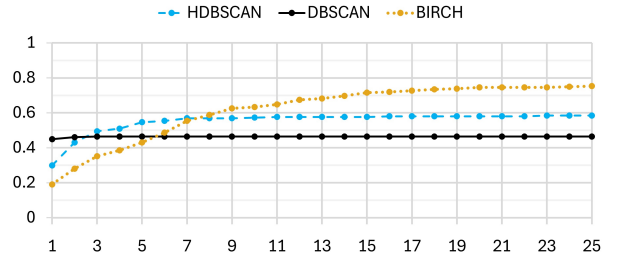
(a) Top-k Precision without a Filter



(b) Top-k Accuracy without a Filter



(c) Top-k Precision: Structural Quality Filter



(d) Top-k Accuracy: Structural Quality Filter

Figure 5: Top-k Precision and Accuracy of the Tests with and without a Filter

5.4 Results

We initially analyse the results of the experiments conducted without applying any filters, followed by a discussion of the results when only using the structurally good clusters. Figure 5 shows all the results for these tests. We use the graphs up to $k = 25$ to clearly show the trends, with the lower k values arguably the most interesting.

We see a similar behaviour in all graphs. The top- k scores increase as k increases, but they plateau at some point. The gain to the score from increasing k becomes too small, so recommending more incidents does not seem appropriate or productive. We note where the techniques plateau (higher is better) and how quickly they approach this level (lower k is better).

5.4.1 No Filter: Figure 5a and Figure 5b show the results for the techniques without a filter applied. With HDBSCAN, top- k precision reaches around 84.05% at $k=10$. At this level, the technique has basically plateaued. Even doubling the number of suggestions to $k=20$ would only increase precision to 85.77%. Completeness is 86.90%, resulting in a Top-10 accuracy of 73.03% and a Top-20 accuracy of 73.78%.

Using DBSCAN results in 54.22% completeness, thus providing fewer suggestions than when using HDBSCAN. top-10 precision is 82.77% while top-20 precision only increases this to 86.66%. This results in top- k accuracy scores of 44.88% and 46.98% for $k=10$ and $k=20$, respectively.

BIRCH underperforms compared to the other two techniques. Although it outputs a suggestion to almost all incidents with around 99.70% completeness, its top- k precision is much lower at both ten and twenty recommendations, with 55.89% and 63.14%, respectively. Consequently, its top- k accuracy follows a similar pattern, reaching 55.72% at $k=10$ and 62.95% at $k=20$. It also does not start plateauing even at twenty recommendations.

5.4.2 Good Structural Quality Filter: Figure 5c and Figure 5d show the results for the techniques with the structural quality filter applied. Using only clusters with good structural quality gives some significant changes. Most notably, filtering yields a sizable improvement in precision for DBSCAN. Top- k precision goes up to 96.10% even at $k=1$, plateauing at $k=3$ with 99.20%. While completeness drops to 46.82%, a total of 7.4% point decrease, this is more than compensated by the improved precision, causing top- k accuracy to increase to 44.94% at $k=1$, plateauing at $k=3$ with 46.44%.

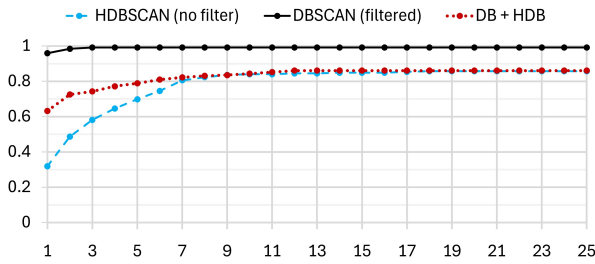
The impact of filtering is lower for the other two techniques. For HDBSCAN, top- k precision remains roughly the same, at around 80.53% for $k=10$ and 81.58% for $k=20$. The completeness decreases considerably, reaching 68.91%, which gives an accuracy of 57.30% at $k=10$ and 58.05% at $k=20$. For BIRCH, filtering improves precision. Top- k precision reaches 72.84% at $k=10$, starting to stabilise at $k=20$ with around 85.77%. Completeness falls to 86.90%, yielding a top- k accuracy reached around 63.30% when $k=10$ and 74.53% when $k=20$. Despite the improvement, the precision score remains lower than the other techniques at lower values of k .

5.4.3 Combined Recommendation System and Pattern Lifespan: In most results, the accuracy plateaus close to ten recommendations. As discussed before, the output of the attribution ranking must be actionable. It should present known attacks possibly related to the actor within a reasonable k . We consider $k=10$ a suitable threshold for the number of related attacks to output for a new incident.

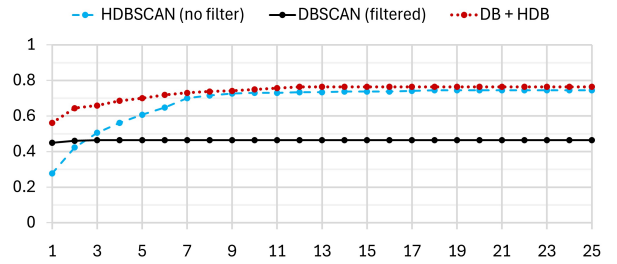
Interpreting the results, we see that the filtered DBSCAN reaches an undisputed precision, although at slightly lower completeness when compared to e.g. HDBSCAN without filtering. Combining these techniques can improve the attribution further. As an example, by combining the most precise and the most complete techniques to optimize the accuracy.

As a combined technique, we take: When a new incident happens, apply the filtered DBSCAN and take its first three recommendations due to its great precision. Then, if there is no recommendation, we use the unfiltered HDBSCAN with a given k . Figure 6 compares the combined approach against the standalone techniques by showing its performance for any k . We consider that $k=10$ has a good balance in this approach. Overall, we get a gain in completeness to 88.76%. At ten recommendations, it reaches 84.38% of precision and 73.78% accuracy. The correct attribution in the first recommendation also increases to more than half of the cases, and the combined approach outperforms the two techniques, especially when using a lower k .

We see that a combination of techniques can outperform the individual recommenders. While beyond the scope of this paper, combinations with techniques based on aspects other than involved domains could provide complementary information and further improve results. Our simple method of combining them relies on one method having a near-perfect precision, but methods exist that do not need to use this assumption. These different methods that combine multiple data types can easily integrate our approach as one of their sources [37].



(a) Top-k Precision: Combining Techniques



(b) Top-k Accuracy: Combining Techniques

Figure 6: Top-k Precision and Accuracy for the Tests Combining Techniques

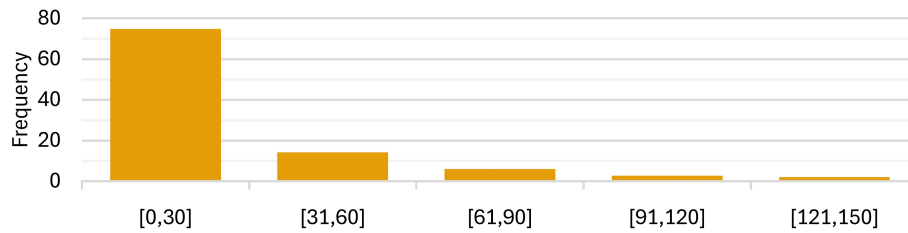


Figure 7: Lifespan of DNS Patterns

In our last test, depicted in Figure 7, we investigated the temporal behaviour of DNS Patterns. We used the combined approach proposed above for this test because it is the most complete. We then applied it to the entire dataset. At first glance, it is apparent that DNS patterns exhibit a brief lifespan, and the most valuable data is within the last 30 days. Specifically, over 55% of them remain active for a mere two weeks, while an overwhelming 95% last for only three months.

In our previous experiments, comparisons used the complete datasets for the known attacks. Considering the temporal nature of incidents, whereby only historical incidents are available whenever during a prediction, this scenario might not be realistic unless attribution happens with some delay. So, we also ran temporal experiments, i.e. assuming that only old attacks are available. As the experiment in Figure 7 shows, DNS patterns last a maximum of 150 days. With that reflection, we use a sliding window that includes only known attacks from the last 150 days for the temporal analysis.

Using the recommended method (DB+HDB) from previous tests with ten recommendations, we see a similar precision of around 86.20% but with a lower completeness at around 68.56%. Although fewer incidents receive a recommendation, the recommendations given have the same reliability as in previous experiments. The only cases where precision suffered greatly was when using the oldest incidents from the datasets, which did not have a knowledge base to learn from. Considering these results, the impression given by the lifespan of DNS Patterns is further confirmed when analysing how much the precision varies when building a dataset from scratch and matching any new attacks observed.

Considering these results, a CSIRT can establish an adequate knowledge base by prioritising the most recent attacks reported by the threat community when analysing a new incident.

6 CONCLUSIONS

In this paper, we proposed a method for threat attribution that uses DNS patterns, making it feasible to analyse them during security incidents. While domains used by attackers change or multiple attackers might use certain domains, such as with malware as a service, the composition of domains used still has enough consistency to be used to attribute attacks. Our results show that it is possible to attribute new incidents with high precision by considering sets of clustered domains and looking at their similarity with already attributed attacks.

The actionability of our method is supported by providing information on these similar known attacks, as well as including their overlaps (in the form of shared clusters) and scores.

A time distance analysis of these similar attacks shows that most are within a few weeks to a few months, limiting the scope for data collection and providing some insight into how long domain resources are reused by attackers.

Combining different clustering techniques already delivers improved accuracy and optimises using the available domain information. Integrating our method with approaches that use other trustworthy information sources is a logical and promising next step. Another possibility is testing other Machine Learning approaches that solve classification problems in the clustering phase.

REFERENCES

- [1] Turki Al lelah, George Theodorakopoulos, Philipp Reinecke, Amir Javed, and Eirini Anthi. 2023. Abuse of Cloud-Based and Public Legitimate Services as Command-and-Control (C&C) Infrastructure: A Systematic Literature Review. *Journal of Cybersecurity and Privacy* 3, 3 (Sept. 2023), 558–590. <https://doi.org/10.3390/jcp3030027> Number: 3 Publisher: Multidisciplinary Digital Publishing Institute.
- [2] Kamal Alieyan, Ammar Almomani, Ahmad Manasrah, and Mohammed M. Kadhun. 2017. A survey of botnet detection based on DNS. *Neural Computing and Applications* 28, 7 (July 2017), 1541–1558. <https://doi.org/10.1007/s00521-015-2128-0>
- [3] Luca Allodi. 2017. Economic Factors of Vulnerability Trade and Exploitation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 1483–1499. <https://doi.org/10.1145/3133956.3133960>
- [4] Giovanni Apruzzese, Pavel Laskov, Edgardo Montes De Oca, Wissam Mallouli, Luis Brdalo Rapa, Athanasios Vasileios Grammatopoulos, and Fabio Di Franco. 2023. The Role of Machine Learning in Cybersecurity. *Digital Threats: Research and Practice* 4, 1 (March 2023), 1–38. <https://doi.org/10.1145/3545574>
- [5] Sean Barnum. 2014. *Standardizing cyber threat intelligence information with the structured threat information expression (stix)*. Technical Report. MITRE.
- [6] Leyla Bilge, Engin Kirda, Christopher Kruegel, and Marco Balduzzi. 2011. EXPOSURE: Finding Malicious Domains Using Passive DNS Analysis. In *18th Annual Network & Distributed System Security Symposium Proceedings*. 1–17.
- [7] Check Point Threat Intelligence & Research. 2016. *Inside Nuclear's Core: Unraveling a Malware-as-a-Service Infrastructure*. Technical Report 2. Check Point Software Technologies. https://blog.checkpoint.com/wp-content/uploads/2016/08/InsideNuclearsCore_UnravelingMalwarewareasaService.pdf
- [8] Richard Clayton and Tony Mansfield. 2014. A Study of Whois Privacy and Proxy Service Abuse. In *DBLP Proceedings*. Springer.
- [9] Christopher Crowley, Barbara Filkins, and John Pescatore. 2023. *SANS 2023 SOC Survey* / SANS Institute. Survey. Escal Institute of Advanced Technologies (SANS). <https://www.sans.org/white-papers/2023-sans-soc-survey/>
- [10] Ron Davidson. 2021. The fight against malware as a service. *Network Security* 2021, 8 (Aug. 2021), 7–11. [https://doi.org/10.1016/S1353-4858\(21\)00088-X](https://doi.org/10.1016/S1353-4858(21)00088-X)
- [11] Brad Duncan. 2016. Understanding Angler Exploit Kit - Part 1: Exploit Kit Fundamentals. <https://unit42.paloaltonetworks.com/unit42-understanding-angler-exploit-kit-part-1-exploit-kit-fundamentals/>
- [12] Guillaume Dupont, Cristoffer Leite, Daniel Ricardo dos Santos, Elisa Costante, Jerry den Hartog, and Sandro Etalle. 2021. Similarity-Based Clustering For IoT Device Classification. In *2021 IEEE International Conference on Omni-Layer Intelligent Systems (COINS)*. 1–7. <https://doi.org/10.1109/COINS51742.2021.9524201>
- [13] Florian J. Egloff and Max Smeets. 2023. Publicly attributing cyber attacks: a framework. *Journal of Strategic Studies* 46, 3 (April 2023), 502–533. <https://doi.org/10.1080/01402390.2021.1895117> Publisher: Routledge _eprint: <https://doi.org/10.1080/01402390.2021.1895117>

- [14] Lorraine Finlay and Christian Payne. 2019. The Attribution Problem and Cyber Armed Attacks. *American Journal of International Law (AJIL)* 113 (Jan. 2019), 202–206. <https://doi.org/10.1017/aju.2019.35>
- [15] Sanjay Goel and Brian Nussbaum. 2021. Attribution Across Cyber Attack Types: Network Intrusions and Information Operations. *IEEE Open Journal of the Communications Society* 2 (2021), 1082–1093. <https://doi.org/10.1109/OJCOMS.2021.3074591>
- [16] Chris Grier, Lucas Ballard, Juan Caballero, Neha Chachra, Christian J. Dietrich, Kirill Levchenko, Panayiotis Mavrommatis, Damon McCoy, Antonio Nappa, Andreas Pitsillidis, Niels Provos, M. Zubair Rafique, Moheeb Abu Rajab, Christian Rosso, Kurt Thomas, Vern Paxson, Stefan Savage, and Geoffrey M. Voelker. 2012. Manufacturing compromise: the emergence of exploit-as-a-service. In *Proceedings of the 2012 ACM conference on Computer and communications security*. ACM, Raleigh North Carolina USA, 821–832. <https://doi.org/10.1145/2382196.2382283>
- [17] Hamed Haddadpajouh, Amin Azmoodeh, Ali Dehghantanha, and Reza M. Parizi. 2020. MVFC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution. *IEEE Access* 8 (2020), 139188–139198. <https://doi.org/10.1109/ACCESS.2020.3012907> Conference Name: IEEE Access.
- [18] Thomas Hyslip and Jason Pittman. 2015. A Survey of Botnet Detection Techniques by Command and Control Infrastructure. *Journal of Digital Forensics, Security and Law* 10, 1 (Jan. 2015). <https://doi.org/10.15394/jdfls.2015.1195>
- [19] Joonas Hämäläinen, Susanne Jauhainen, and Tommi Kärkkäinen. 2017. Comparison of Internal Clustering Validation Indices for Prototype-Based Clustering. *Algorithms* 10, 3 (Sept. 2017), 105. <https://doi.org/10.3390/a10030105> Number: 3 Publisher: Multidisciplinary Digital Publishing Institute.
- [20] Ehtsham Irshad and Abdul Basit Siddiqui. 2023. Cyber threat attribution using unstructured reports in cyber threat intelligence. *Egyptian Informatics Journal* 24, 1 (March 2023), 43–59. <https://doi.org/10.1016/j.eij.2022.11.001>
- [21] Amir Namavar Jahromi, Hadis Karimipour, Ali Dehghantanha, and Kim-Kwang Raymond Choo. 2021. Toward Detection and Attribution of Cyber-Attacks in IoT-Enabled Cyber-Physical Systems. *IEEE Internet of Things Journal* 8, 17 (Sept. 2021), 13712–13722. <https://doi.org/10.1109/IJOT.2021.3067667>
- [22] Joint Task Force Transformation Initiative. 2011. *Managing information security risk : organization, mission, and information system view*. Technical Report NIST SP 800-39. National Institute of Standards and Technology, Gaithersburg, MD. NIST SP 800-39 pages. <https://doi.org/10.6028/NIST.SP.800-39> Edition: 0.
- [23] Christos Karapapas, Iakovos Pittaras, Nikos Fotiou, and George C. Polyzos. 2020. Ransomware as a Service using Smart Contracts and IPFS. In *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, Toronto, ON, Canada, 1–5. <https://doi.org/10.1109/ICBC48266.2020.9169451>
- [24] Kyoungmin Kim, Youngsup Shin, Justin Lee, and Kyungho Lee. 2021. Automatically Attributing Mobile Threat Actors by Vectorized ATT&CK Matrix and Paired Indicator. *Sensors* 21, 19 (Jan. 2021), 6522. <https://doi.org/10.3390/s21196522> Number: 19 Publisher: Multidisciplinary Digital Publishing Institute.
- [25] Frankie Li. 2014. *APT Attribution and DNS Profiling*. Technical Report. Black Hat. <https://infocondb.org/con/black-hat/black-hat-usa-2014/apt-attribution-and-dns-profiling>
- [26] Chaoyi Lu, Baojun Liu, Yiming Zhang, Zhou Li, Fenglu Zhang, Haixin Duan, Ying Liu, Joann Qiongna Chen, Jinjin Liang, Zaifeng Zhang, Shuang Hao, and Min Yang. 2021. From WHOIS to WHOWAS: A Large-Scale Measurement Study of Domain Registration Privacy under the GDPR. In *Proceedings 2021 Network and Distributed System Security Symposium*. Internet Society, Virtual. <https://doi.org/10.14722/ndss.2021.23134>
- [27] Willi Mann, Nikolaus Augsten, and Panagiotis Bouras. 2016. An empirical evaluation of set similarity join techniques. *Proceedings of the VLDB Endowment* 9, 9 (May 2016), 636–647. <https://doi.org/10.14778/2947618.2947620>
- [28] Leland McInnes, John Healy, and Steve Astels. 2017. hdbSCAN: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (March 2017), 205. <https://doi.org/10.21105/joss.00205>
- [29] Per Håkon Meland, Yara Fared Fahmy Bayoumy, and Guttorm Sindre. 2020. The Ransomware-as-a-Service economy within the darknet. *Computers & Security* 92 (May 2020), 101762. <https://doi.org/10.1016/j.cose.2020.101762>
- [30] Asaf Nadler, Avi Aminov, and Asaf Shabtai. 2019. Detection of malicious and low throughput data exfiltration over the DNS protocol. *Computers & Security* 80 (Jan. 2019), 36–53. <https://doi.org/10.1016/j.cose.2018.09.006>
- [31] Gonzalo Navarro. 2001. A guided tour to approximate string matching. *Comput. Surveys* 33, 1 (March 2001), 31–88. <https://doi.org/10.1145/375360.375365>
- [32] Umara Noor, Zahid Anwar, Tehmina Amjad, and Kim-Kwang Raymond Choo. 2019. A machine learning-based FinTech cyber threat attribution framework using high-level indicators of compromise. *Future Generation Computer Systems* 96 (July 2019), 227–242. <https://doi.org/10.1016/j.future.2019.02.013>
- [33] Lior Perry, Bracha Shapira, and Rami Puzis. 2019. NO-DOUBT: Attack Attribution Based On Threat Intelligence Reports. In *2019 IEEE International Conference on Intelligence and Security Informatics (ISI)*. 80–85. <https://doi.org/10.1109/ISI.2019.8823152>
- [34] Pieter Philippaerts and Wouter Joosen. 2023. *Cyberdreigingslandschap: Een blik op 2024*. Technical Report. KOMPAS.
- [35] Stijn Pletinckx, Cyril Trap, and Christian Doerr. 2018. Malware Coordination using the Blockchain: An Analysis of the Cerber Ransomware. In *2018 IEEE Conference on Communications and Network Security (CNS)*. 1–9. <https://doi.org/10.1109/CNS.2018.8433199>
- [36] Daniel Plohmman, Khaled Yakdan, Michael Klatt, Elmar Gerhards-Padilla, and Johannes Bader. 2016. A Comprehensive Measurement Study of Domain Generating Malware. In *USENIX Proceedings*.
- [37] Li Qiang, Yang Zeming, Liu Baoxu, Jiang Zhengwei, and Yan Jian. 2017. Framework of Cyber Attack Attribution Based on Threat Intelligence. In *Interoperability, Safety and Security in IoT*, Nathalie Mitton, Hakima Chaouchi, Thomas Noel, Thomas Watteyne, Alban Gabillon, and Patrick Capolisi (Eds.). Springer International Publishing, Cham, 92–103. https://doi.org/10.1007/978-3-319-52727-7_11
- [38] Yitong Ren, Yanjun Xiao, Yinghai Zhou, Zhiyong Zhang, and Zhihong Tian. 2023. CSKG4APT: A Cybersecurity Knowledge Graph for Advanced Persistent Threat Organization Attribution. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (June 2023), 5695–5709. <https://doi.org/10.1109/TKDE.2022.3175719> Conference Name: IEEE Transactions on Knowledge and Data Engineering.
- [39] Thomas Rid and Ben Buchanan. 2015. Attributing Cyber Attacks. *Journal of Strategic Studies* 38, 1-2 (Jan. 2015), 4–37. <https://doi.org/10.1080/01402390.2014.977382> Publisher: Routledge. eprint: <https://doi.org/10.1080/01402390.2014.977382>
- [40] Dilip Sahoo. 2022. Cyber Threat Attribution with Multi-View Heuristic Analysis. In *Handbook of Big Data Analytics and Forensics*, Kim-Kwang Raymond Choo and Ali Dehghantanha (Eds.). Springer International Publishing, Cham, 53–73. https://doi.org/10.1007/978-3-030-74753-4_4
- [41] Doyen Sahoo, Chenghao Liu, and Steven C. H. Hoi. 2019. Malicious URL Detection using Machine Learning: A Survey. <https://doi.org/10.48550/arXiv.1701.07179> arXiv:1701.07179 [cs].
- [42] Erich Schubert, Jörg Sander, Martin Ester, Hans Peter Kriegel, and Xiaowei Xu. 2017. DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN. *ACM Transactions on Database Systems* 42, 3 (Sept. 2017), 1–21. <https://doi.org/10.1145/3068335>
- [43] Silvia Sebastián, Raluca-Georgia Diugan, Juan Caballero, Iskander Sanchez-Rola, and Leyla Bilge. 2023. Domain and Website Attribution beyond WHOIS. In *Annual Computer Security Applications Conference*. ACM, Austin TX USA, 124–137. <https://doi.org/10.1145/3627106.3627190>
- [44] Paulo Shakarian, Gerardo I. Simari, Geoffrey Moores, and Simon Parsons. 2015. Cyber Attribution: An Argumentation-Based Approach. In *Cyber Warfare: Building the Scientific Foundation*, Sushil Jajodia, Paulo Shakarian, V.S. Subrahmanian, Vipin Swarup, and Cliff Wang (Eds.). Springer International Publishing, Cham, 151–171. https://doi.org/10.1007/978-3-319-14039-1_8
- [45] Amit Sharma, Brij B Gupta, Awadhesh Singh, and V. Saraswat. 2023. Advanced Persistent Threats (APT): evolution, anatomy, attribution and countermeasures. *Journal of Ambient Intelligence and Humanized Computing* 14 (May 2023), 1–27. <https://doi.org/10.1007/s12652-023-04603-y>
- [46] Florian Skopik and Timea Pahi. 2020. Under false flag: using technical artifacts for cyber attack attribution. *Cybersecurity* 3, 1 (March 2020), 8. <https://doi.org/10.1186/s42400-020-00048-4>
- [47] Aditya K. Sood and Sherali Zeadally. 2016. A Taxonomy of Domain-Generation Algorithms. *IEEE Security & Privacy* 14, 4 (July 2016), 46–53. <https://doi.org/10.1109/MSP.2016.76> Conference Name: IEEE Security & Privacy.
- [48] Xinjie Sun and Zhifang Liu. 2023. Domain generation algorithms detection with feature extraction and Domain Center construction. *PLOS ONE* 18, 1 (Jan. 2023), e0279866. <https://doi.org/10.1371/journal.pone.0279866>
- [49] Matthew Thomas and Aziz Mohaisen. 2014. Kindred domains: detecting and clustering botnet domains using DNS traffic. In *Proceedings of the 23rd International Conference on World Wide Web*. ACM, Seoul Korea, 707–712. <https://doi.org/10.1145/2567948.2579359>
- [50] Tarun Trivedi, Vinod Parihar, Manas Khatua, and B. M. Mehtre. 2019. Threat Intelligence Analysis of Onion Websites Using Sublinks and Keywords. In *Emerging Technologies in Data Mining and Information Security (Advances in Intelligent Systems and Computing)*, Ajith Abraham, Paramartha Dutta, Jyotsna Kumar Mandal, Abhishek Bhattacharya, and Soumi Dutta (Eds.). Springer, Singapore, 567–578. https://doi.org/10.1007/978-981-13-1501-5_50
- [51] Grigorios Tsoumakas and Ioannis Katakis. 2007. Multi-label classification: An overview. *International Journal of Data Warehousing and Mining* 3, 3 (Jan. 2007), 1–13. <https://doi.org/10.4018/jdwm.2007070101>
- [52] Arun Warikoo. 2021. The Triangle Model for Cyber Threat Attribution. *Journal of Cyber Security Technology* 5, 3-4 (Oct. 2021), 191–208. <https://doi.org/10.1080/23742917.2021.1895532>
- [53] Caixia Yan, Minnan Luo, Huan Liu, Zhihui Li, and Qinghua Zheng. 2018. Top-k multi-class SVM using multiple features. *Information Sciences* 432 (March 2018), 479–494. <https://doi.org/10.1016/j.ins.2017.08.004>
- [54] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. 1996. BIRCH: an efficient data clustering method for very large databases. *ACM SIGMOD Record* 25, 2 (June 1996), 103–114. <https://doi.org/10.1145/235968.233324>
- [55] Yury Zhauniarovich, Issa Khalil, Ting Yu, and Marc Dacier. 2019. A Survey on Malicious Domains Detection through DNS Data Analysis. *Comput. Surveys* 51, 4 (July 2019), 1–36. <https://doi.org/10.1145/3191329>